

Arduino Tutorial

Table of Contents

Table of Contents.....	1
1. Programming Learning.....	2
Before You Begin: Important Notes.....	2
1.1 Install Arduino IDE.....	2
1.2 Install ESP32 Development Environment for Arduino IDE.....	4
1.3 Install the Library File.....	7
1.4 Install CH340 USB driver.....	8
Install the Driver on Windows system	9
Install the Driver on Linux system.....	9
Install the Driver on MAC system.....	10
1.5 Code Examples.....	11
1.5.1 Onboard buzzer.....	11
1.5.2 To Drive a Servo.....	14
1.5.3 Read the Value of the Joystick.....	16
1.5.4 Joystick Control Robot (With Motion Record & Repeat function).....	17
1.5.5 Read the Value of the Web APP.....	20
1.5.6 Using the Robot Arm.....	22
2. Burn Firmware.....	23
2.1 Burning Tool.....	24
2.2 eArm Firmware (Factory Firmware).....	24
2.2.1 Burning eArm Firmware.....	25
2.3 Joystick Control Firmware (with Recording Function).....	26
2.3.1 Burning Joystick Control Firmware.....	27
3. QA.....	29
3.1 Unable to recognize USB serial ports.....	29
3.2 Robotic arm doesn't work.....	29
3.3 Other problems.....	29
4. Contact Us.....	29

Programming Learning

Before You Begin: Important Notes

The ESP32 control board comes preloaded with firmware that allows you to operate the robotic arm as described in the Assembly and User Guide. The preloaded firmware will remain on the board until you upload new code.

Important Notes Before Uploading Code:

- 1: The ESP32 board always runs the most recently uploaded program.
- 2: Each new upload overwrites the previously uploaded program.

This chapter will guide you through programming the robot using code examples ranging from basic to advanced. The final example includes the source code for the factory-preloaded firmware, allowing you to restore the original robotic arm functionality at any time.

1.1 Install Arduino IDE

The latest version of Arduino IDE can be downloaded from the Arduino official website: <https://www.arduino.cc/en/software>

In the following we will demonstrate the installation of **Windows Win 10 and newer, 64 bits** version IDE on PC.



Arduino IDE 2.3.2

The new major release of the Arduino IDE is faster and even more powerful! In addition to a more modern editor and a more responsive interface it features autocompletion, code navigation, and even a live debugger.

For more details, please refer to the [Arduino IDE 2.0 documentation](#).

Nightly builds with the latest bugfixes are available through the section below.

SOURCE CODE

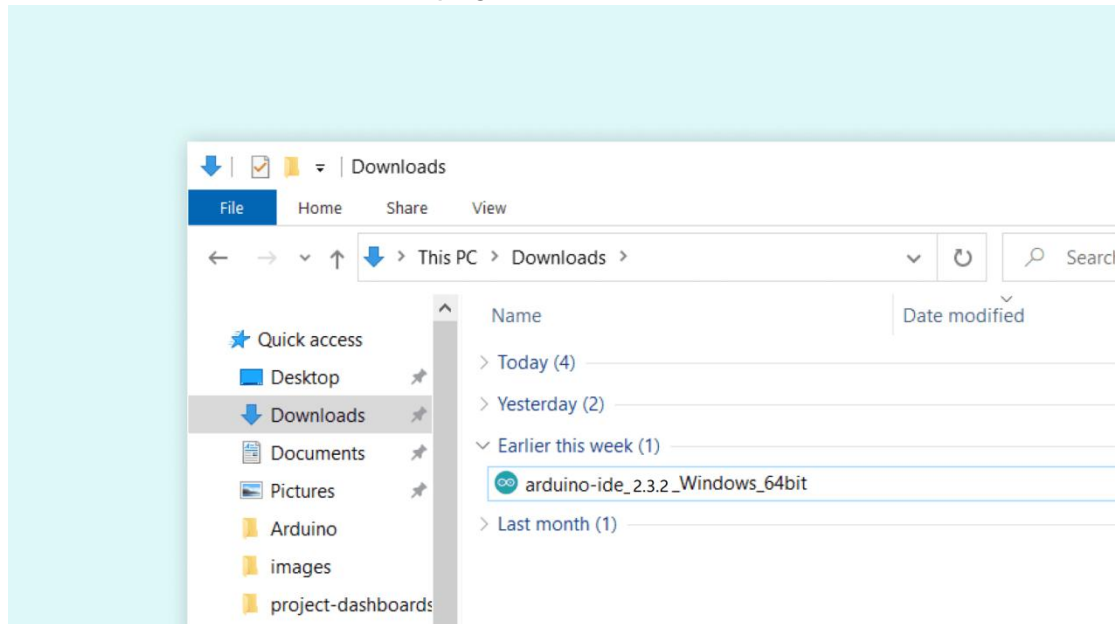
The Arduino IDE 2.0 is open source and its source code is hosted on [GitHub](#).

DOWNLOAD OPTIONS

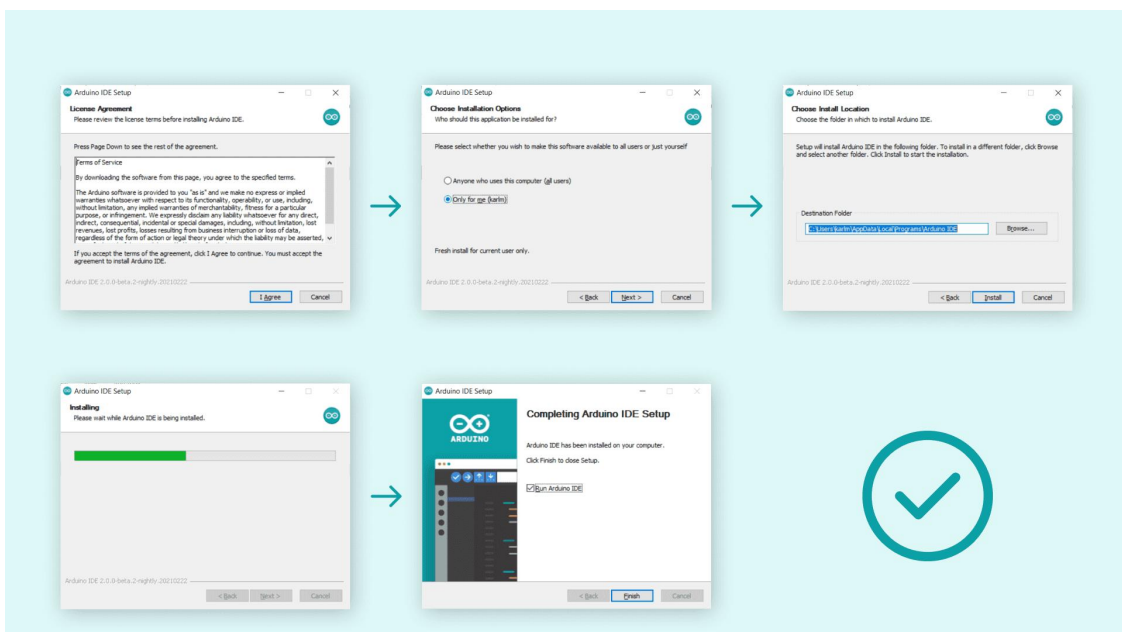
- Windows** Win 10 and newer, 64 bits
- Windows** MSI installer
- Windows** ZIP file
- Linux** AppImage 64 bits (X86-64)
- Linux** ZIP file 64 bits (X86-64)
- macOS** Intel, 10.15: "Catalina" or newer, 64 bits
- macOS** Apple Silicon, 11: "Big Sur" or newer, 64 bits

[Release Notes](#)

To install the Arduino IDE 2.x.x on a Windows computer, simply run the file downloaded from the software page.



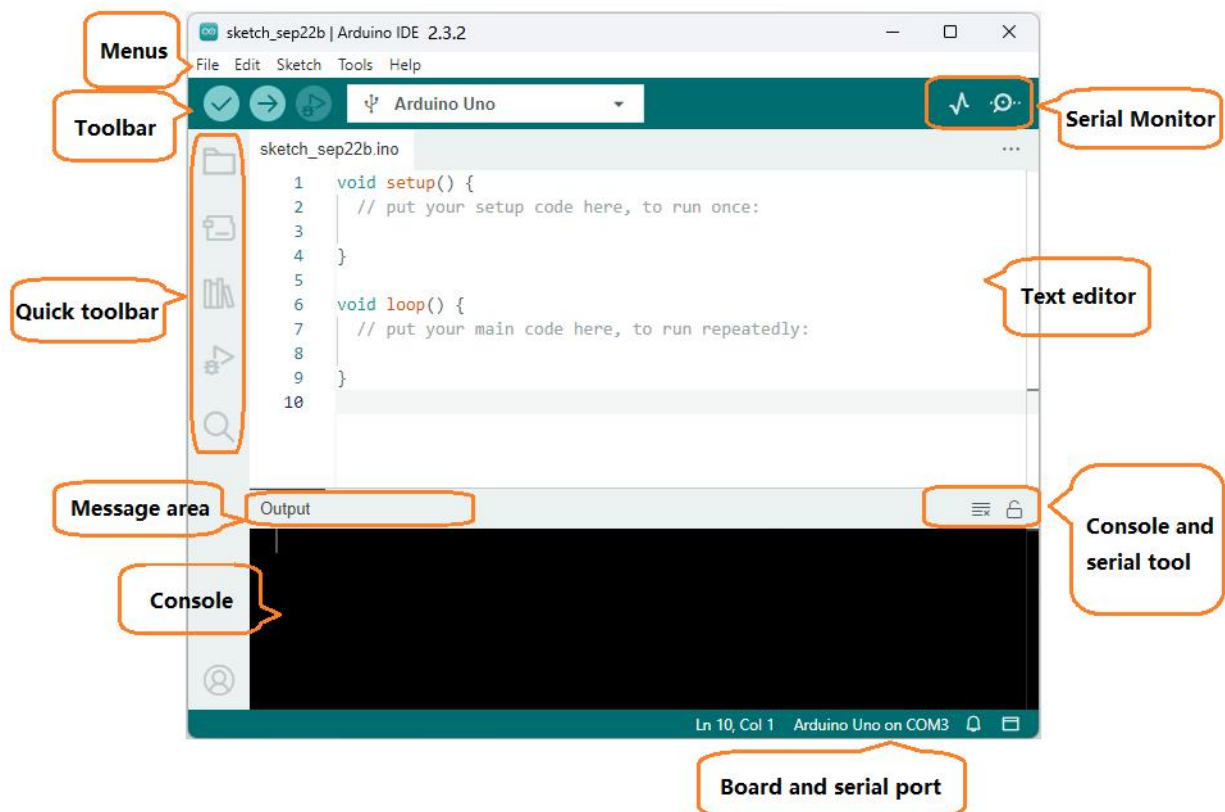
Follow the pop-up window prompts to complete the installation process.



Startup icon:

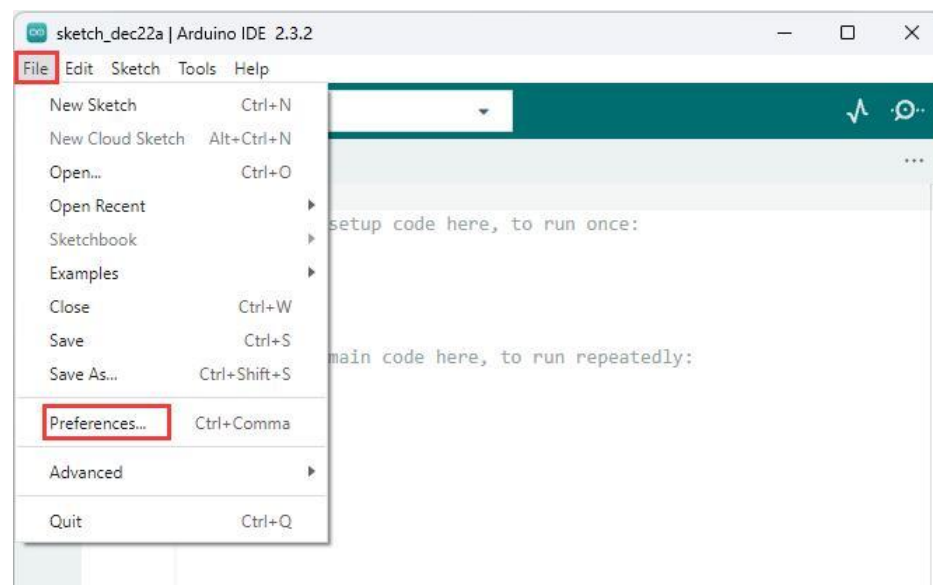


The homepage of Arduino IDE



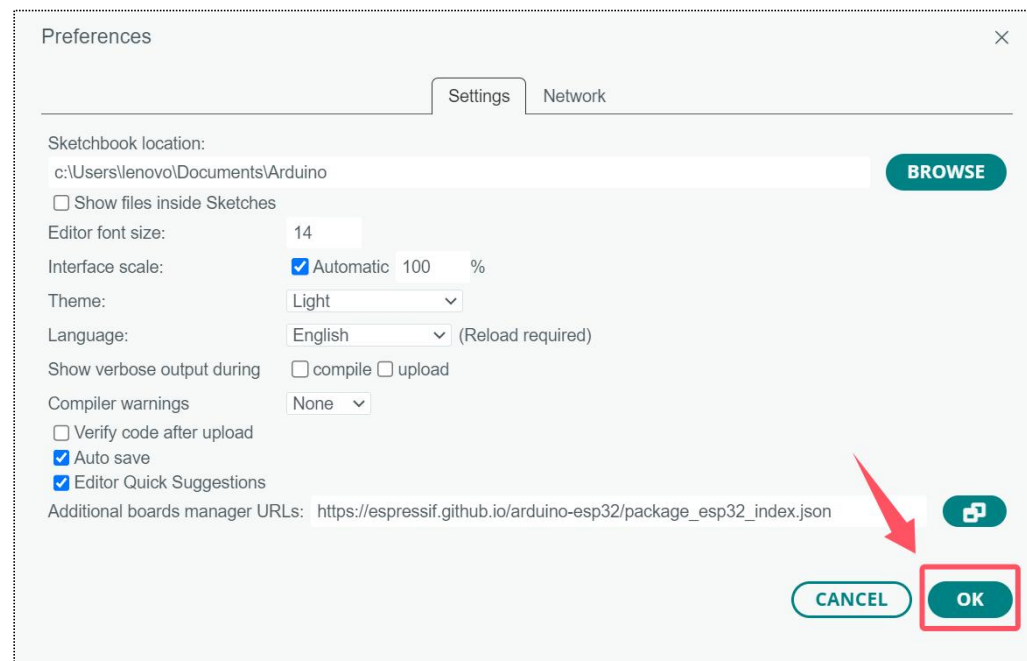
1.2 Install ESP32 Development Environment for Arduino IDE

Open the Arduino IDE, click "File" > "Preferences", as shown below:

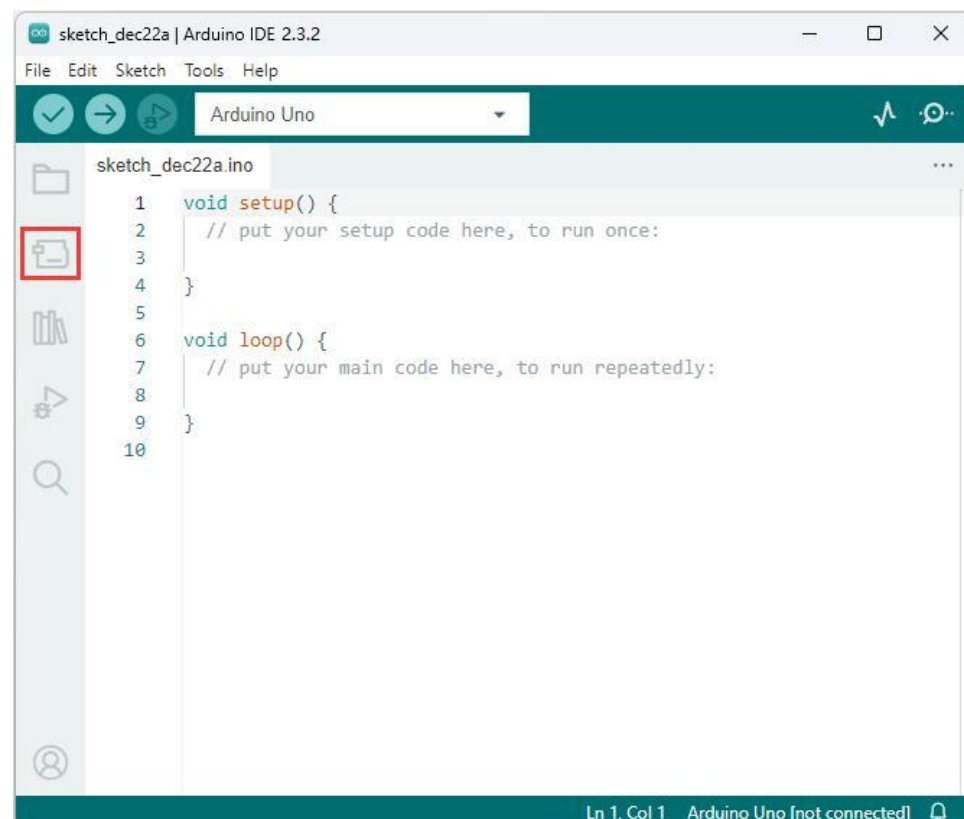


Scroll down the pop-up window and copy the link below into the **Additional boards manager URLs** text box. Then click “OK”.

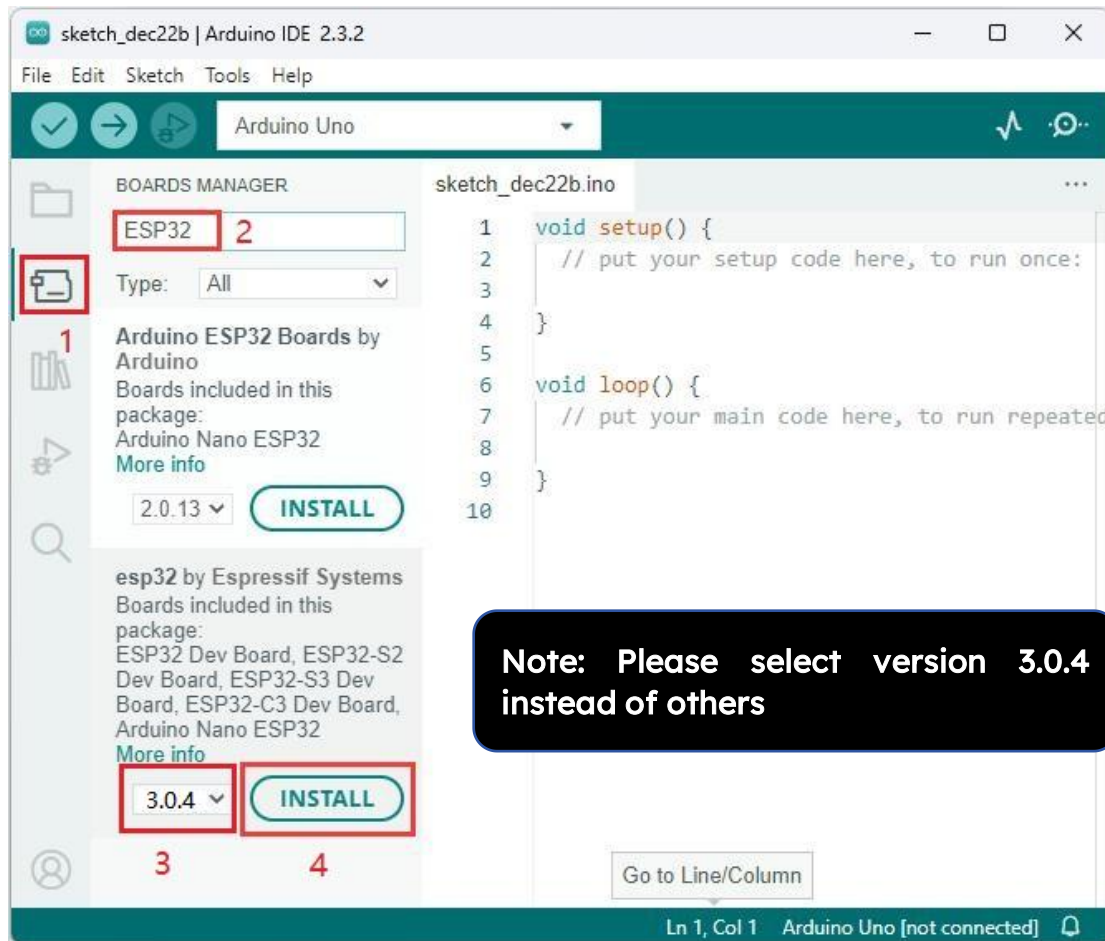
https://espressif.github.io/arduino-esp32/package_esp32_index.json



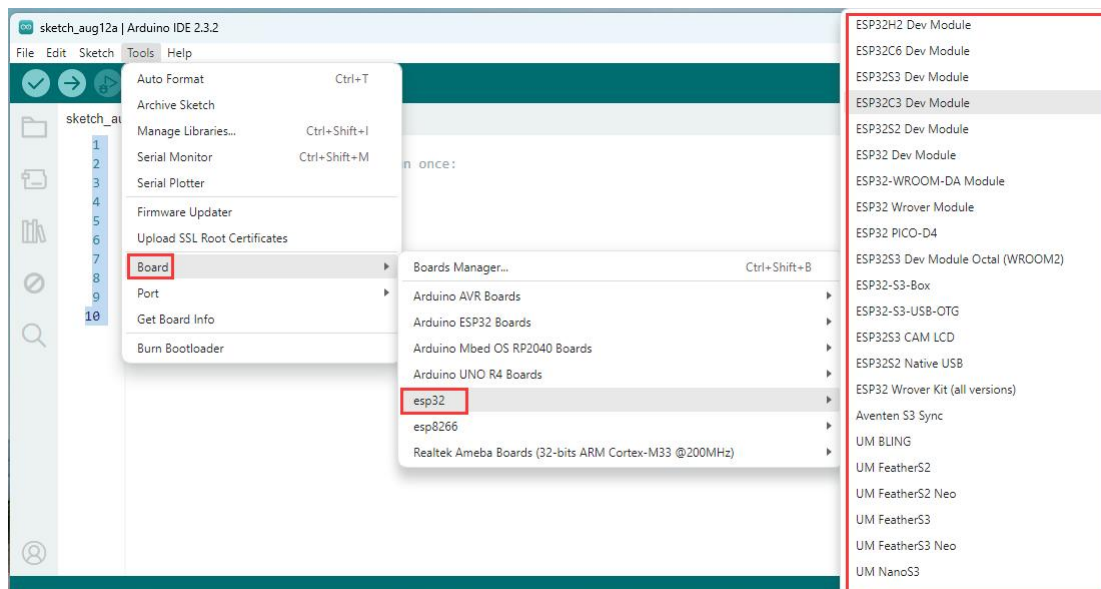
Click “Boards Manager”:



Search for **ESP32** in the Boards Manager, select **esp32 by Espressif Systems** in **3.0.4** version and click **INSTALL**



After successfully installing the ESP32 board, click the "Tools"->"Board"->"ESP32" menu, and select **ESP32C3 Dev Module**.



1.3 Install the Library File

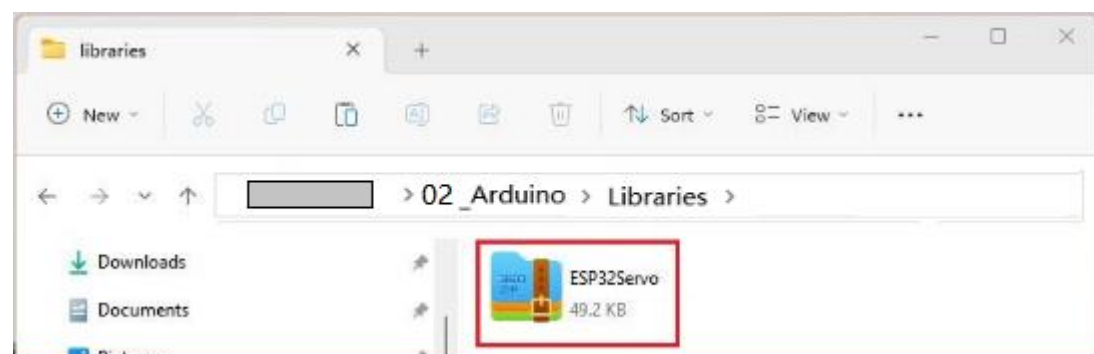
Method 1:

Open the library manager of Arduino IDE and type "**ESP32Servo**" in the search bar, select the library file named ESP32Servo and version **3.0.5** and click Install.

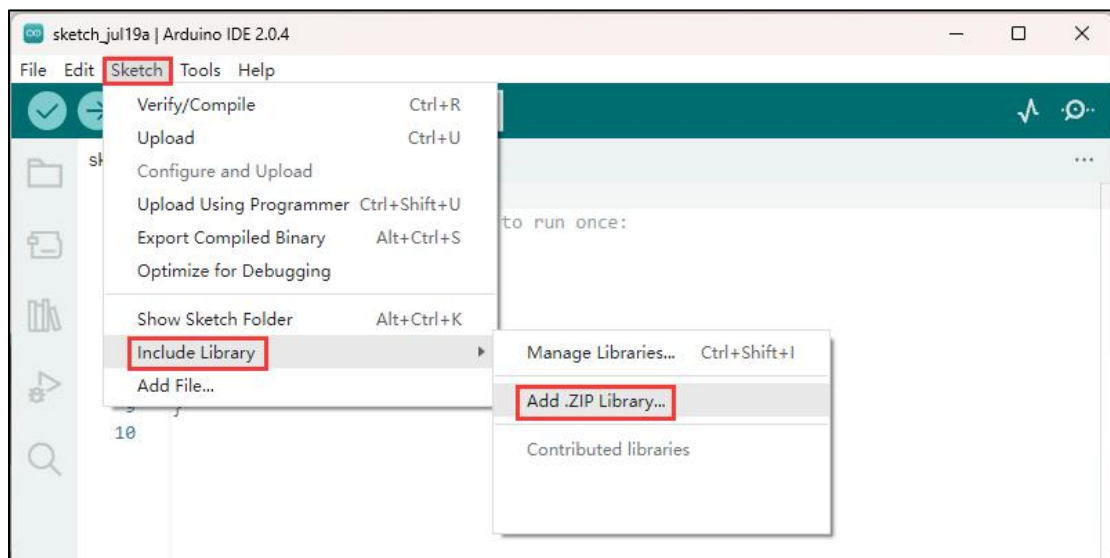


Method 2:

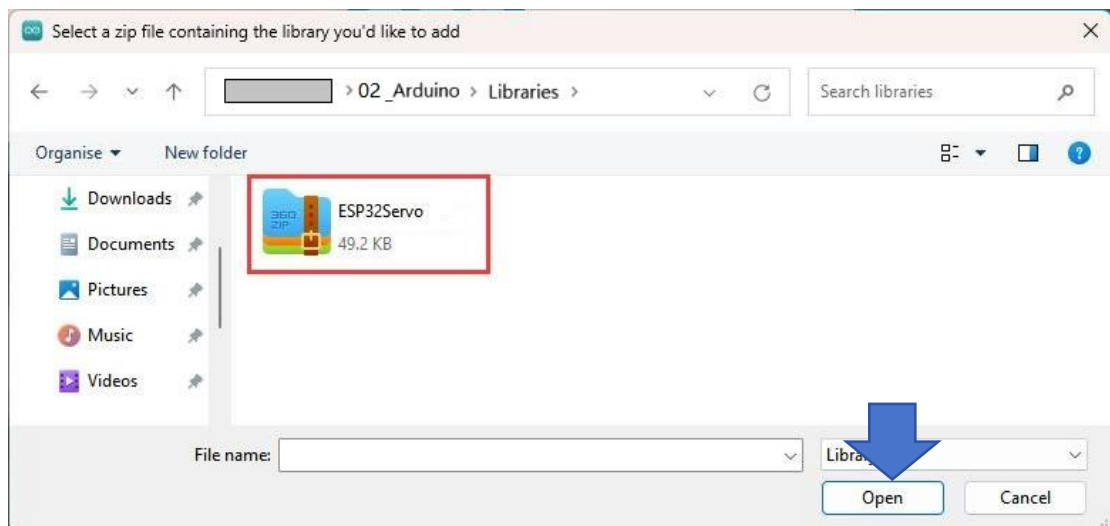
We provide a library zip file in the tutorial package, in the following folder:



Import the library zip file from the Arduino IDE as follows:

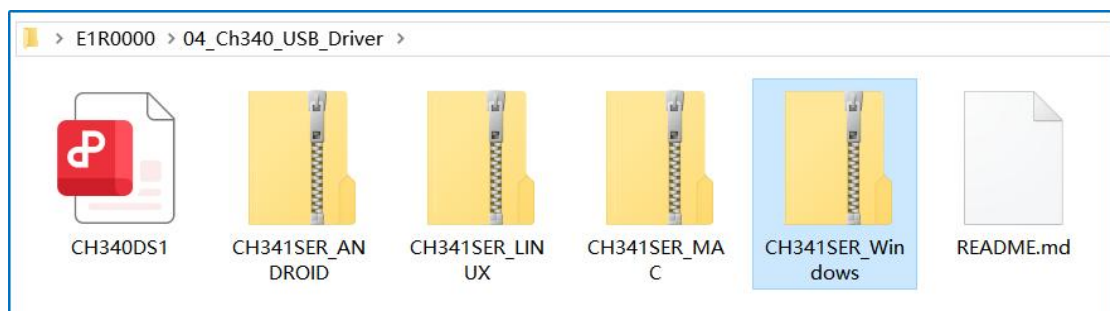


Click **Open** to install the library file.



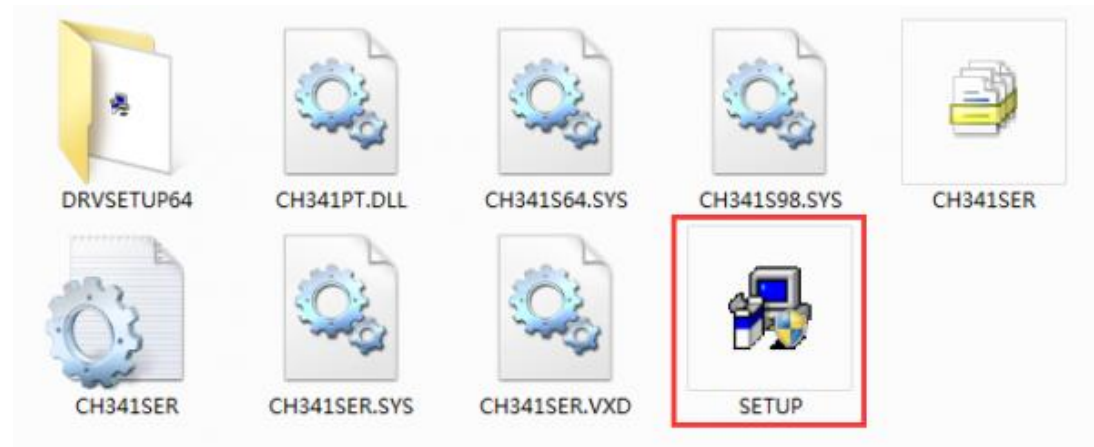
1.4 Install CH340 USB driver

Driver file is provided in our tutorial package:



Install the Driver on Windows system

Unzip the file we provided and double-click **“SETUP”** to run the installer:



If the driver is successfully installed, when the esp32 board is connected to the computer through the USB cable, under the path **“Computer” -> “Properties” -> “Device manager”**, you can find **“USB-SERIAL CH340 (COM3)”**, as shown below:

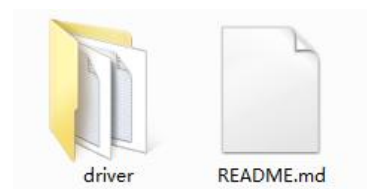


Note: Driver display symbol is **“USB-SERIAL CH340 (COMx)”**, **“x”** can be any number, it depends on what number your computer assigns to the ESP32 device.

Install the Driver on Linux system

Drivers are almost certainly built into your Linux kernel already and it will probably just work as soon as you plug it in. If not you can download the Linux CH340 Driver (but I'd recommend just upgrading your Linux install so that you get the “built in” one).

If you must install it by yourself, check the **“README.md”** file in the zip package.



Install the Driver on MAC system

Link to download the driver:

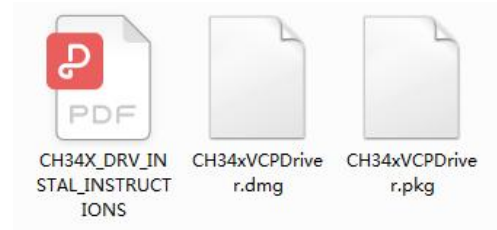
http://www.wch-ic.com/downloads/CH341SER_EXE.html

 download

relation files

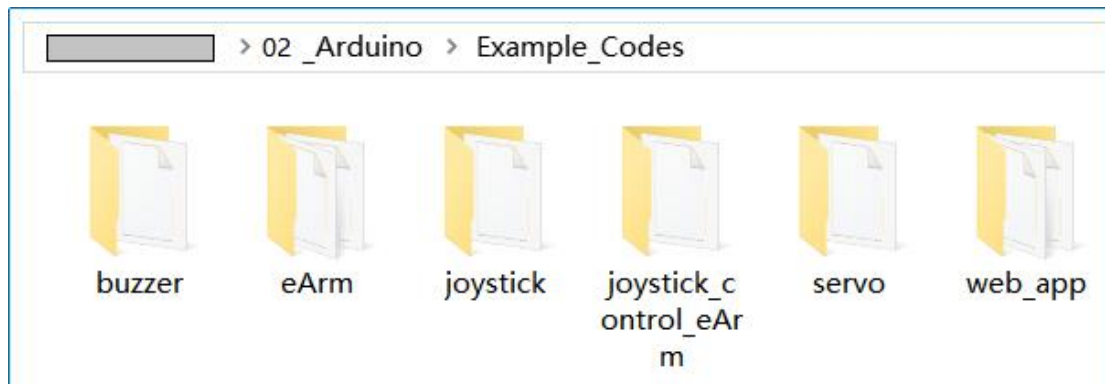
file name	file content
CH341SER.ZIP	CH340/CH341 USB to serial port Windows driver, supports Windows XP/Vista/7/8/8.1/10/11/ SERVER 2003/2008/2012/2016/2019/2022 -32/64bit, Microsoft WHQL Certified, supports USB to 3-line and 9-line serial port.
CH341SER_LINUX.ZIP	Linux driver for USB to serial port, supports CH340 and CH341, supports 32/64-bit operating systems.
CH341SER_MAC.ZIP	For CH340/CH341/CH342/CH343/CH344/CH347/CH9101/CH9102/CH9103/CH9104/CH9143, USB to serial port VCP vendor driver of macOS , supports OS X 10.9~10.15 , OS X 11 (Big Sur) and above , contains installation guide documents.
CH340DS1.PDF	CH340 datasheet, USB bus converter chip which converts USB to serial port, printer port, IrDA SIR etc. Integrated clock, supports various operating systems, chip information can be customized, this datasheet is about USB to serial port and USB to Infrared Adapter SIR.
CH341DS1.PDF	CH341 datasheet, USB bus converter chip with multiple communication interfaces, such as USB to serial port/parallel port/printer port/I2C interface etc. Drivers support for Windows/Linux/Android/Mac, etc. The datasheet is the description of USB to serial port and printer port.
CH341SER_ANDROID.ZIP	CH340/CH341/CH342/CH343/CH344/CH347/CH9101/CH9102/CH9103/CH9104/CH9143 USB to serial port Android driver-free application library and software, for Android OS 4.4 and above in USB Host mode, without loading Android kernel driver and without root access operation. Includes apk installer, lib library file (Java Driver), App Demo routines (USB to UART Demo project SDK).

Download the driver from the website and unzip the file to a local installation directory. You will get a PDF installation guide and two installation packages in different formats. For details, see the PDF installation guide.



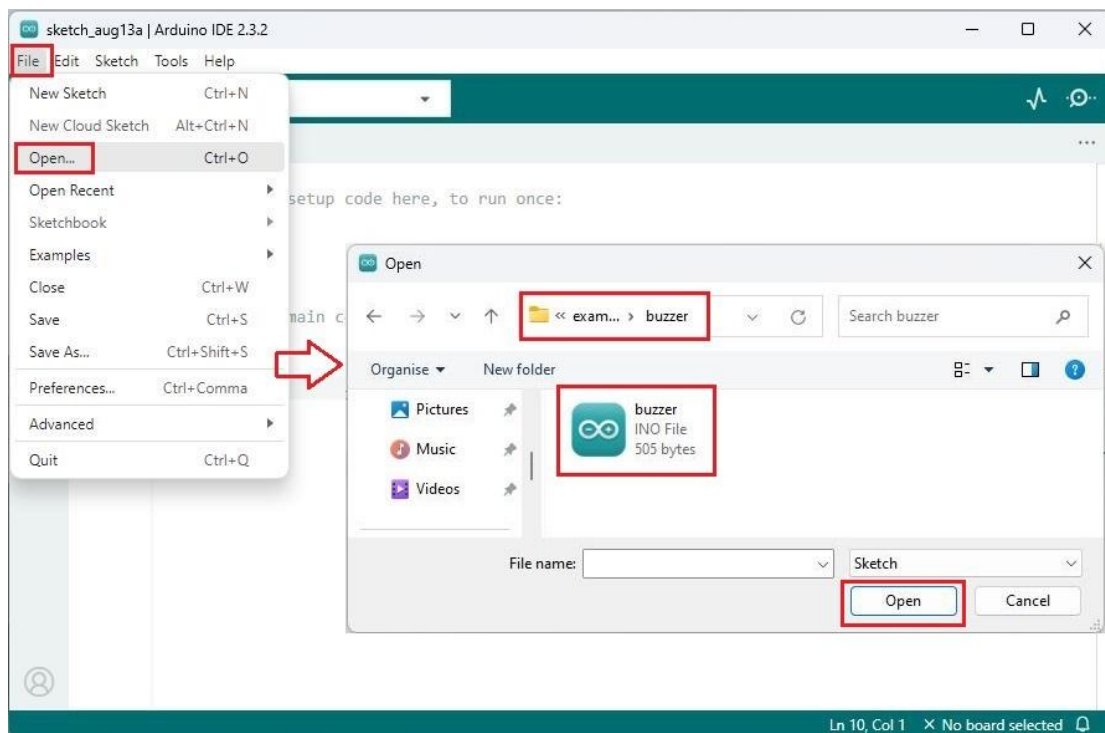
1.5 Code Examples

The sample codes are stored in the following folders in the tutorial package:

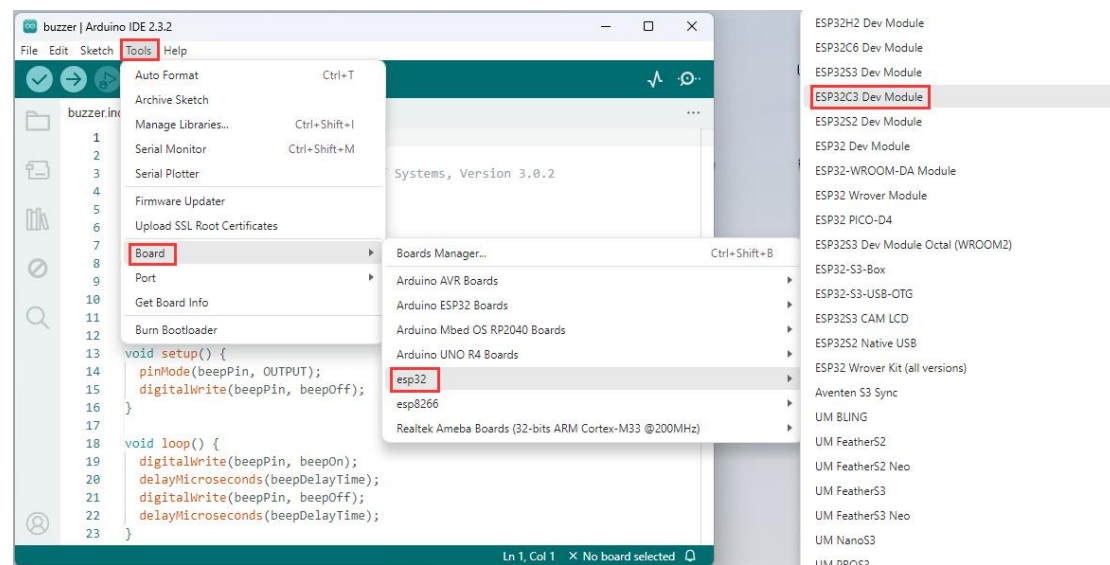


1.5.1 Onboard buzzer

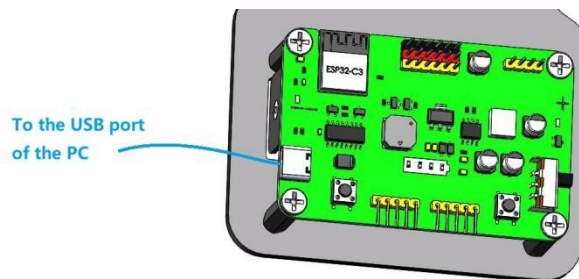
Open the "buzzer" example code in the Arduino IDE:



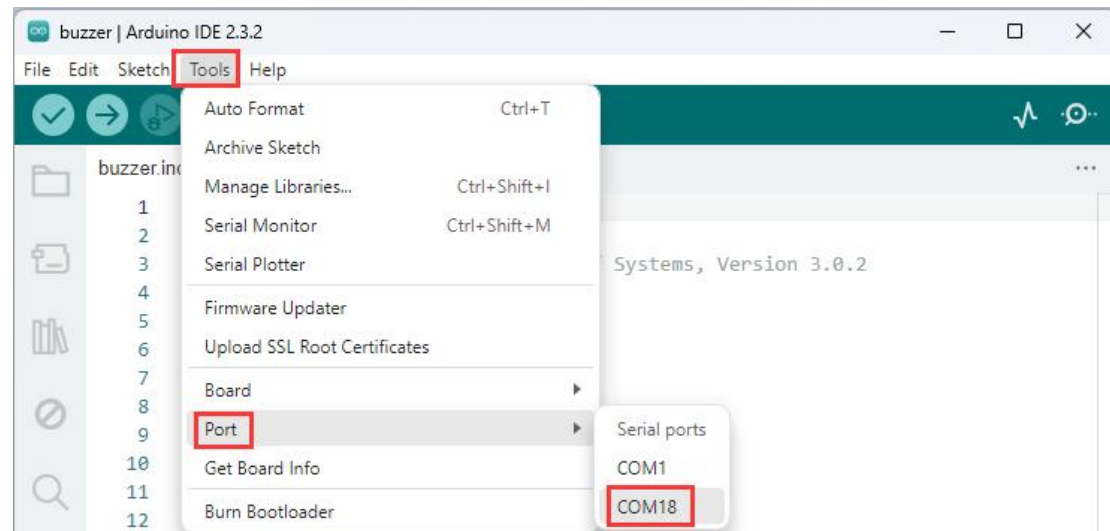
Select board type (**ESP32C3 Dev Module**) :



Use the USB cable to connect the PC and eArm, as shown below:

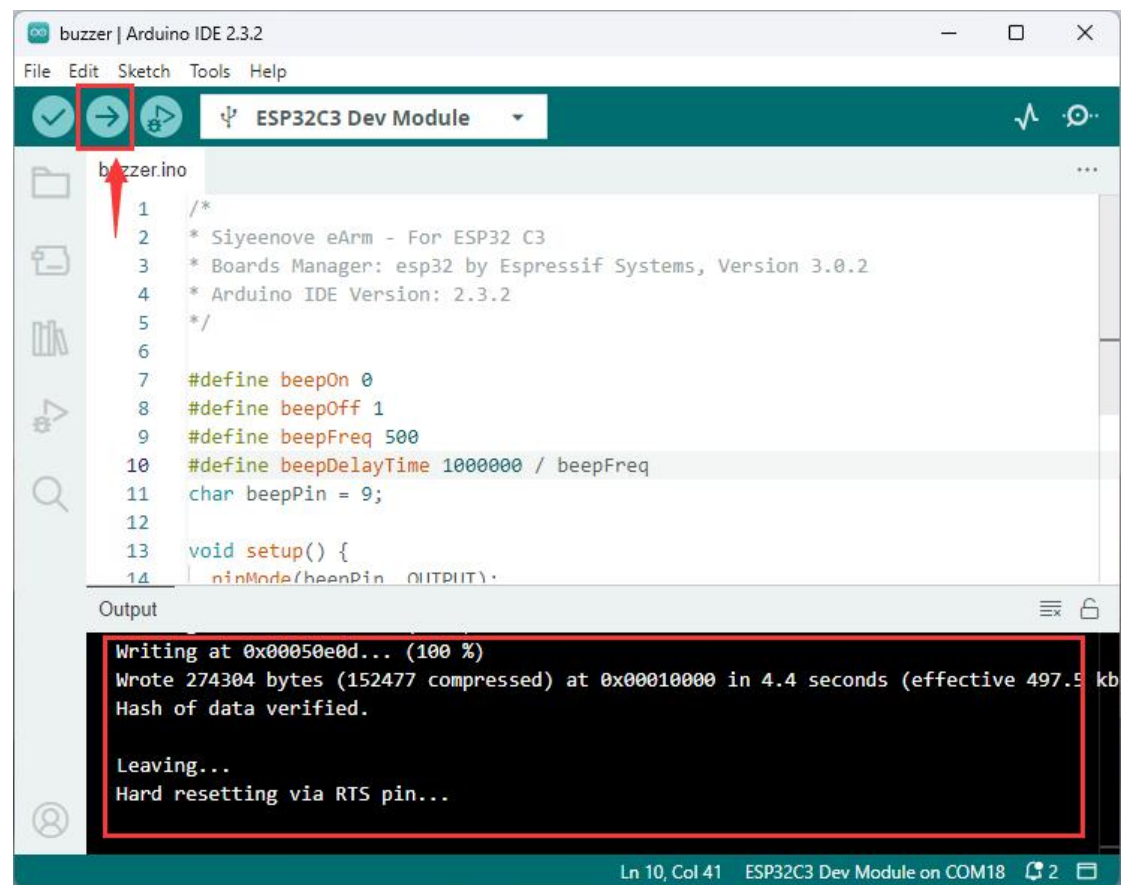


Select Port:

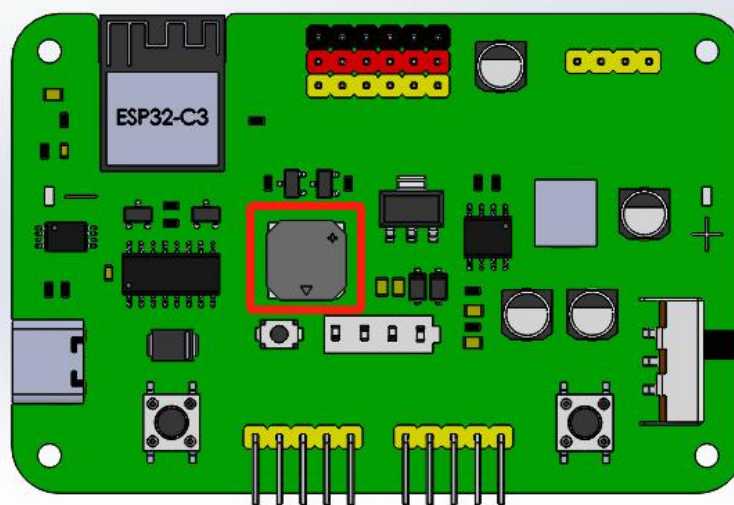


Note: The port in the above picture is COM18. Your computer may have a different numbered port. Please select it according to the port assigned by the device manager on your computer.

Click to **upload** the code:



After the code is uploaded successfully, the buzzer on the ESP32 board will keep making sounds:

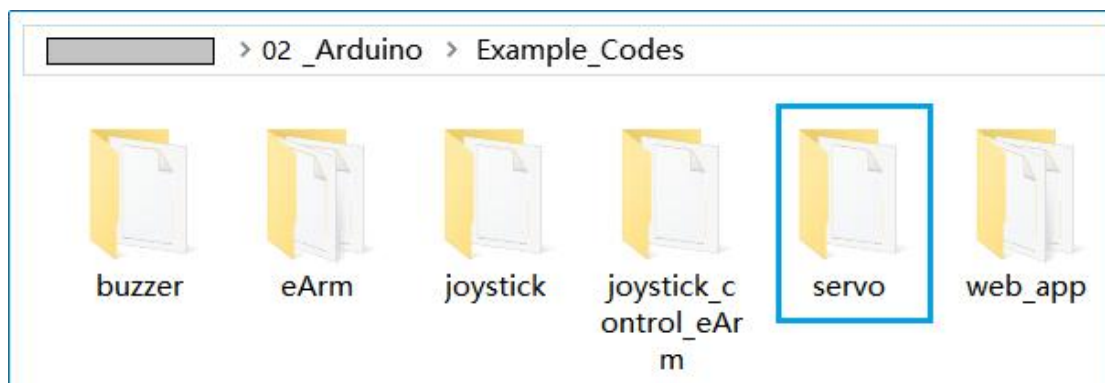


Code Explanation:

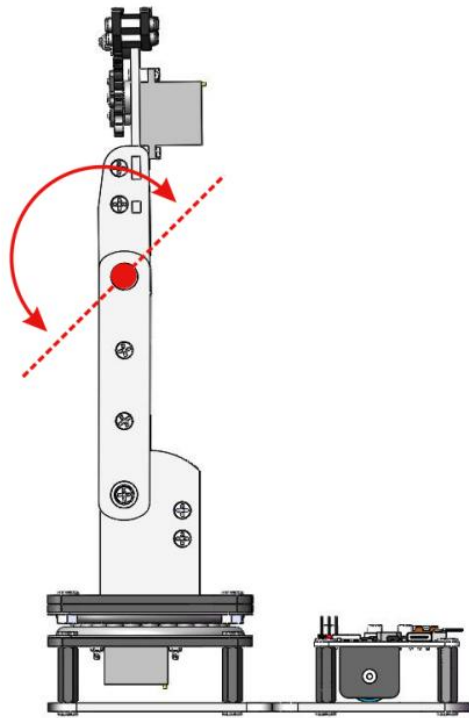
<code>#define beepOn 0</code>	Define a constant "beepOn" with a value of 0.
<code>#define beepDelayTime 1000000 / beepFreq</code>	Define a constant "beepDelayTime" with a value of: 1000000 / beepFreq.
<code>char beepPin = 9;</code>	Define a character variable "beepPin" with a value of 9, mapped to the control pin of the buzzer.
<code>void setup() { ... }</code>	Built-in function of Arduino IDE, only runs once when powered on.
<code>pinMode(beepPin, OUTPUT);</code>	Set pin 9 to output mode.
<code>digitalWrite(beepPin, beepOff);</code>	Pin 9 outputs a high level to turn off the buzzer.
<code>void loop() { ... }</code>	Built-in function of Arduino IDE, runs in a loop all the time when powered on.
<code>delayMicroseconds(beepDelayTime);</code>	Built-in function of Arduino IDE, delay in microseconds.

1.5.2 To Drive a Servo

Please follow the chapter 1.5.1 to upload the "servo" code to the esp32 control board:



After the code is uploaded successfully, the eArm's servo C will swing 0-180, 180-0 in a cycle:

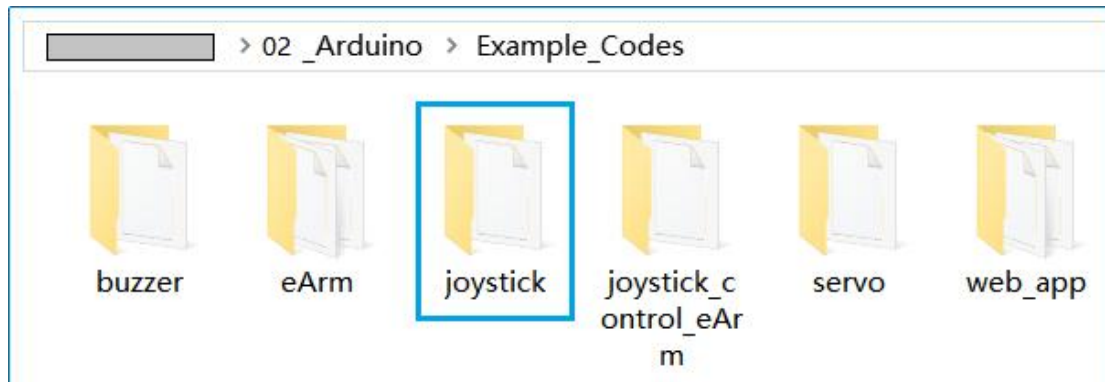


Code Explanation:

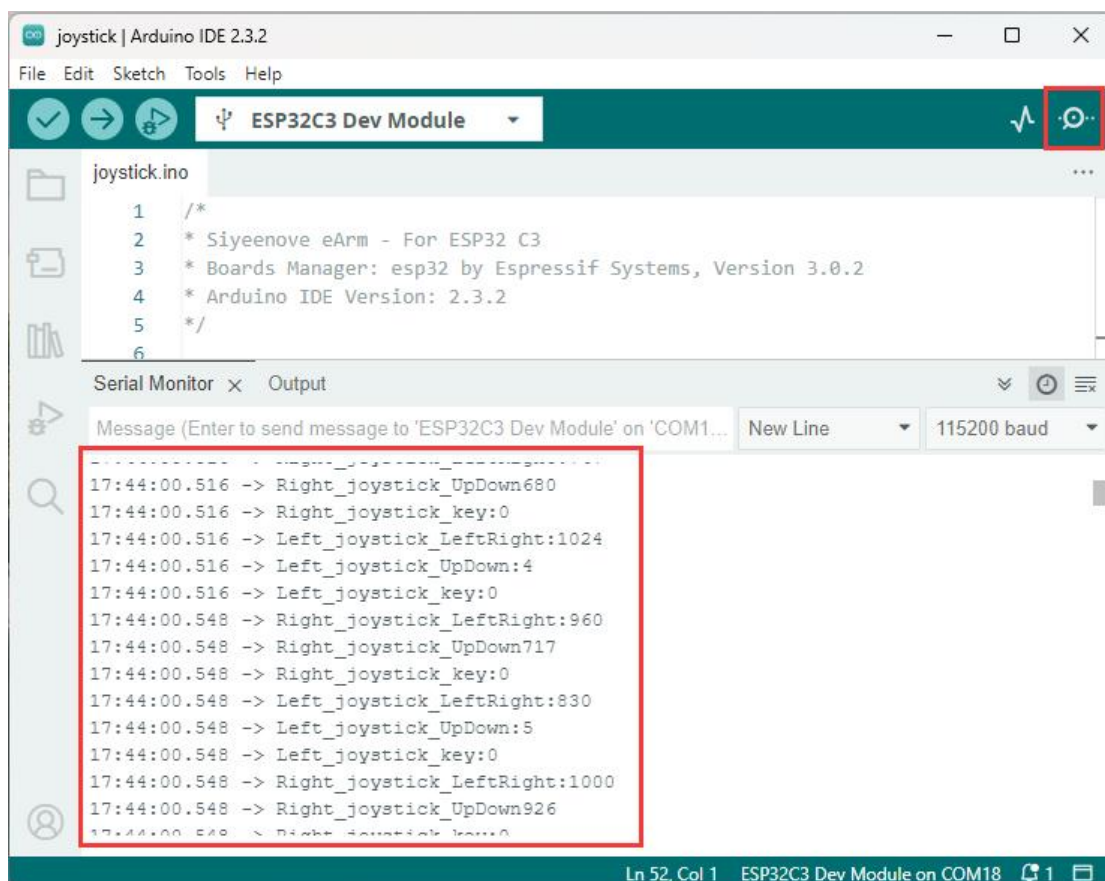
<code>#include <ESP32Servo.h></code>	Include an ESP32 servo driver library.
<code>Servo Aservo;</code>	Define a servo class to drive Servo1.
<code>ESP32PWM::allocateTimer(0);</code>	Allow the servo driver library to use timer 0.
<code>Aservo.setPeriodHertz(50);</code>	Set the pulse drive frequency of the driving servo Aservo to 50 Hz.
<code>Aservo.attach(AservoPin, 500, 2400);</code>	Set the driving servo Aservo to use the AservoPin pin to drive, the latest pulse width is 500 microseconds, and the maximum pulse width is 2400 microseconds.
<code>Aservo.write(aServoAngle);</code>	Set the driving servo Aservo to rotate to the angle aServoAngle.
<code>for(statement1;condition;statement2) { ... }</code>	For loop statement, each time statement1 is executed, it determines whether the condition is true. If so, the statements in the curly braces are executed, otherwise the loop is interrupted; Then statement2 is executed, and the condition is determined again, and the loop is repeated.

1.5.3 Read the Value of the Joystick

Please follow the chapter 1.5.1 to upload the "joystick" code to the esp32 control board:



Then open the serial port monitor of Arduino IDE, shake the handle left, right, up, down, or press the joystick down, and the serial port will print the joystick value:

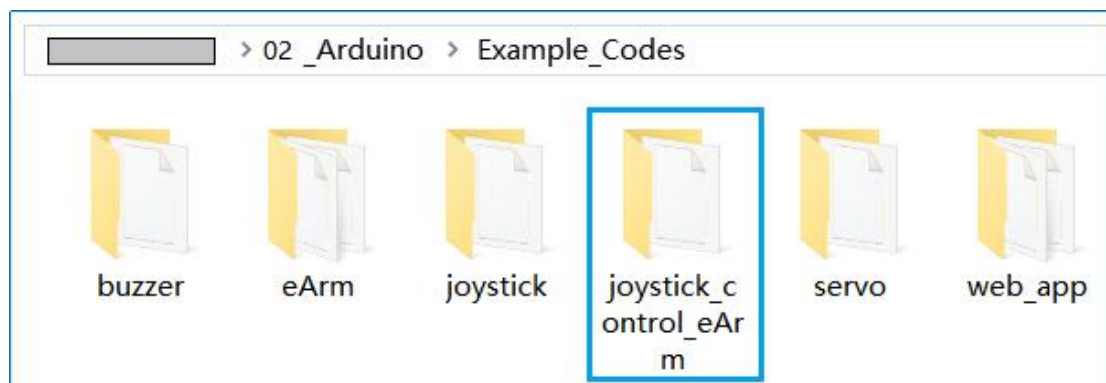


Code Explanation:

<code>Serial.begin(115200);</code>	Set the serial port baud rate to 115200
<code>pinMode(left_joystick_KeyPin, INPUT);</code>	Set the key pin of the left joystick to "left_joystick_KeyPin" and the mode to output.
<code>pinMode(right_joystick_KeyPin, INPUT_PULLUP);</code>	Set the key pin of the right joystick to "right_joystick_KeyPin", the mode to output, and with a pull-up resistor.
<code>Serial.print("Left_joystick_LeftRight:");</code>	The serial port prints a string without line breaks.
<code>Serial.println(analogRead(left_joystick_LeftRightPin));</code>	The serial port prints analog values and wraps.
<code>analogRead(left_joystick_LeftRightPin);</code>	Read the value of the left joystick shaking left and right.
<code>delay(1000);</code>	Delay 1000 milliseconds.

1.5.4 Joystick Control Robot (With Motion Record & Repeat function)

Please follow the chapter 1.5.1 to upload the "joystick_control_eArm" code to the esp32 control board:



Joystick Control – Robotic Arm Operation Guide:

A+: Controls Aservo, robotic arm rotates left.	A-: Controls Aservo, robotic arm rotates right.
B+ : Controls Bservo, robotic arm big arm swings forward.	B- : Controls Bservo, robotic arm big arm swings backward.
C+: Controls Cservo, robotic arm small arm swings forward.	C-: Controls Cservo, robotic arm small arm swings backward.
D+: Controls Dservo, robotic arm gripper opens (loosens).	D-: Controls Dservo, robotic arm gripper closes (tightens).
E: Record Action – Press E button once to record one action (up to 20 actions can be recorded in sequence). Each press triggers a short beep. When the maximum recording capacity is reached, the buzzer emits one long beep.	F : Playback — After recording your sequence, press the F button once to start looping the recorded actions continuously. The buzzer will beep once, and the arm will repeat the actions in a loop. To exit the loop, press and hold the F button (long press); the buzzer will emit one long beep to indicate playback has stopped.

Note: Because dynamic memory is used, the eArm will lose all recorded actions after power-off!

You can also modify the default maximum number of recorded actions through the code:

```

53  const int act_max=10000;    //Default 20 action,4 the Angle of servo
54  int act[act_max][4];        //Only can change the number of action
55  int num=0,num_do=0;

```

Output

```

cmd /c IF 0==1 COPY /y "C:\\Users\\11235\\AppData\\Local\\Arduino15\\packages\\esp32\\tools\\openocd
cmd /c IF 0==1 COPY /y "C:\\Users\\11235\\AppData\\Local\\Arduino15\\packages\\esp32\\hardware\\esp3
cmd /c IF 0==1 COPY /y "C:\\Users\\11235\\AppData\\Local\\Arduino15\\packages\\esp32\\hardware\\esp3

Using library ESP32Servo at version 3.0.5 in folder: C:\\Users\\11235\\Documents\\Arduino\\libraries\\ESP3
"C:\\Users\\11235\\AppData\\Local\\Arduino15\\packages\\esp32\\tools\\riscv32-esp-elf-gcc\\esp-2021r
Sketch uses 251604 bytes (19%) of program storage space. Maximum is 1310720 bytes.
Global variables use 174428 bytes (53%) of dynamic memory, leaving 153252 bytes for local variables.

```

After compiling the code in Arduino IDE, check the output log. When the number of recorded actions is changed to 10,000, it only occupies 53% of the dynamic memory, which means you can still record even more actions!

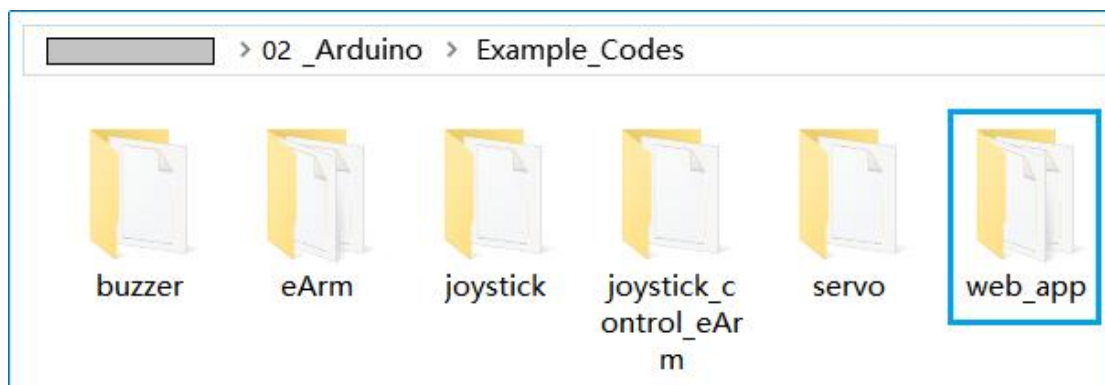
Code Explanation:

<code>eArm arm;</code>	Defines a robotic arm class.
<code>arm.ServoAttach(4,5,6,7);</code>	The A, B, C, and D servos of the robotic arm are connected to pins 4, 5, 6, and 7 of the control board respectively.
<code>arm.JoystickAttach(0,1,10,2,3,8);</code>	The X, Y, and Z pins of the left joystick are connected to pins 0, 1, and 10 of the control board respectively. The X, Y, and Z pins of the right joystick are connected to pins 2, 3, and 8 of the control board respectively.
<code>int act[x][y];</code>	Defines a two-dimensional integer array, where x is the row parameter and y is the column parameter.
<code>arm.down(speed);</code>	The robotic arm swings downward, where speed is the swing speed.
<code>arm.up(speed);</code>	The robotic arm swings upward, where speed is the swing speed.
<code>arm.left(speed);</code>	The robotic arm rotates to the left, where speed is the swing speed.
<code>arm.right(speed);</code>	The robotic arm rotates to the right, where speed is the swing speed.
<code>arm.clawClose(speed);</code>	The robotic arm gripper closes, where speed is the swing speed.
<code>arm.clawOpen(speed);</code>	The robotic arm gripper opens, where speed is the swing speed.
<code>void turnUD(void);</code>	Joystick controls the robotic arm to swing up and down.
<code>void turnLR(void);</code>	Joystick controls the robotic arm to rotate left and right.
<code>void claw(void);</code>	Joystick controls the robotic arm gripper to open and close.
<code>void date_processing(int *x,int *y);</code>	Processes the X and Y axis data of the joystick and reads the maximum swing data of the axes.
<code>void buzzer(int H,int L);</code>	Buzzer function for the joystick Z-axis buttons. H is the time the buzzer is on (in microseconds); L is the time the buzzer is off (in microseconds).
<code>arm.JoystickL.read_z();</code>	Reads the Boolean value of the left joystick Z-axis.
<code>arm.JoystickR.read_z();</code>	Reads the Boolean value of the right joystick Z-axis.
<code>void Re_action(void);</code>	Records the current actions of the robotic arm and

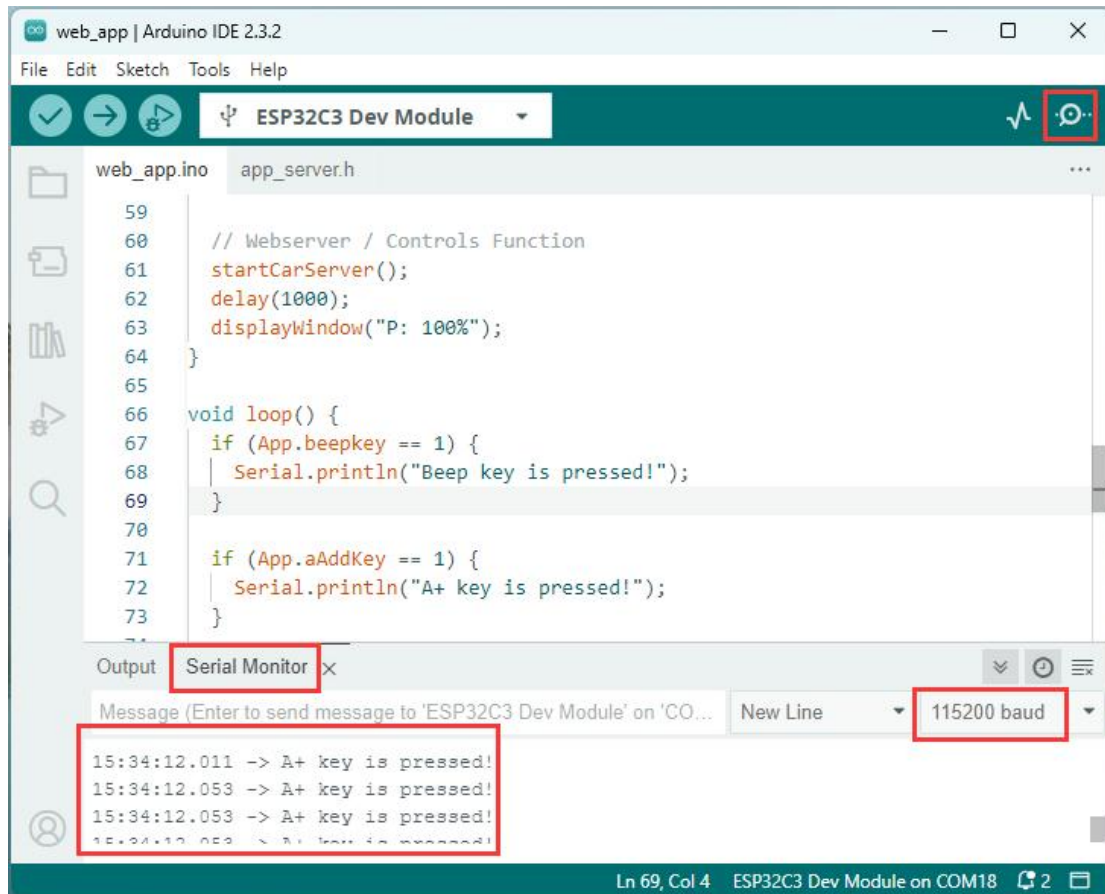
	saves them in the two-dimensional array.
<code>void Ex_action(void);</code>	Executes all actions recorded in the two-dimensional array.
<code>void executionAction(int *angle,int speed);</code>	Executes a single action, where *angle is a pointer and speed is the execution speed.
<code>arm.JoyStickL.read_x();</code>	Reads the data of the left joystick X-axis. Return value: 0-4059.
<code>arm.JoyStickL.read_y();</code>	Reads the data of the left joystick Y-axis. Return value: 0-4059.
<code>arm.JoyStickR.read_x();</code>	Reads the data of the right joystick X-axis. Return value: 0-4059.
<code>arm.JoyStickR.read_y();</code>	Reads the data of the right joystick Y-axis. Return value: 0-4059.

1.5.5 Read the Value of the Web APP

Please follow the chapter 1.5.1 and upload the "web_app" code to the esp32 control board:



Follow the "**Assembly and User Guide**" chapter to connect "eArm" to wifi and use the web app, then open the serial monitor of Arduino IDE and click the button of the web app. The serial monitor will print the string corresponding to the button:



Code Explanation:

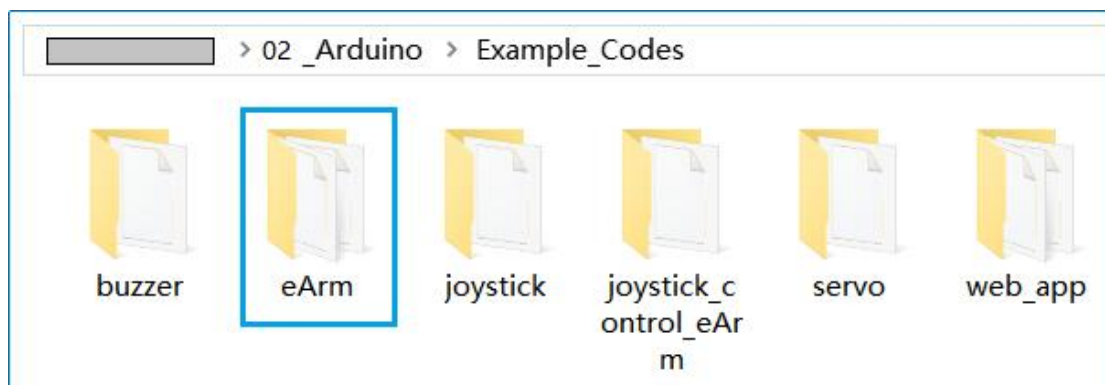
<code>#include "app_server.h"</code>	Include the "app_server.h" header file from the same folder.
<code>bool ap = 1;</code>	Define a Boolean variable "ap" and assign it a value of 1.
<code>const char* ssid = "eArm";</code>	Define a constant character pointer and assign a value to it.
<code>int channel = 11;</code>	Define an integer variable "channel" and assign it a value of 11.
<code>WiFi.mode(WIFI_STA);</code>	Set Wi-Fi to "station" mode.
<code>WiFi.begin(ssid, password);</code>	Set the Wi-Fi name and password, and start Wi-Fi.
<code>while(condition) {...}</code>	While conditional loop statement, if the condition is true, the loop executes the statements in the curly braces; otherwise, the loop is interrupted.
<code>WiFi.status();</code>	Read the wifi connection status.
<code>WiFi.localIP();</code>	Read the wifi IP address in "station" mode.
<code>WiFi.mode(WIFI_AP);</code>	Set the wifi to AP mode.
<code>WiFi.softAP(ssid, password, channel, hidden, maxconnection);</code>	In AP mode, set the wifi name, password, channel, whether to hide the name, and the maximum number of connections.

<code>WiFi.softAPIP();</code>	Read the IP address of the wifi in AP mode.
<code>startCarServer();</code>	Start the web server.
<code>displayWindow("P: 100%");</code>	Display the string on the front end of the web page.
<code>if(condition){ ... }</code>	If the condition is true, the statements in the curly braces are executed; otherwise, they are not executed.
<code>App.beepkey</code>	The state variable value of the buzzer button on the front end of the web page.

1.5.6 Using the Robot Arm

This sample code is the complete source code of the factory firmware. Uploading it to the control board will restore the original factory functionality.

Please follow the chapter 1.5.1 to upload the "eArm" code to the esp32 control board:



After uploading the eArm code, you can now control this robotic arm as described in the previous section: [Assembly and User Guide > Using the Robot Arm](#).

Code Explanation:

<code>TaskHandle_t TASK_HandleOne = NULL;</code>	Create a thread handle.
<code>xTaskCreate(...);</code>	Initialize the thread.
<code>void TASK_ONE(void* param) { ... }</code>	Thread function, which runs in parallel with the loop() function.

2

Burn Firmware

Providing the device's "factory reset program" directly to users in the form of a firmware file (e.g., .bin or .hex), allowing them to flash it into the device without setting up an Arduino development environment, thereby completing the restoration process.

1. Issues with the Traditional Approach

Typically, restoring a device via Arduino requires users to:

- ▶ Install the Arduino IDE or related development tools.
- ▶ Download the source code (.ino file), possibly requiring additional library setups and dependencies.
- ▶ Compile and upload it to the device.

This process has a high barrier for non-technical users and may fail due to environment-related issues.

2. Advantages of Providing Firmware Directly

Simplified User Process:

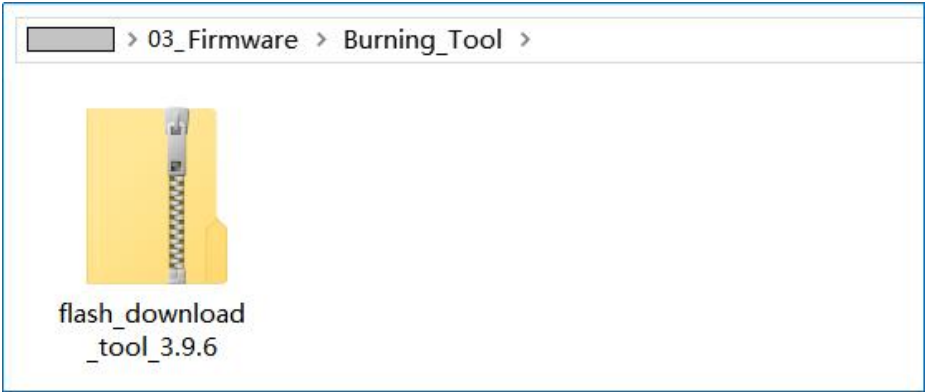
- ▶ Provide a pre-compiled firmware file (e.g., firmware.bin), allowing users to flash it with a simple tool (e.g., a serial flasher) in one step.
- ▶ No need to install Arduino IDE, resolve compilation errors, or manage library conflicts.

Use Cases:

- ▶ Restoring a malfunctioning device to factory settings.
- ▶ Mass-flashing the same firmware to multiple devices (improves efficiency).

2.1 Burning Tool

Burning Tool is stored in the following folder:



You can also download the latest version Burning Tool from the official website:

<https://www.espressif.com/en/support/download/other-tools>

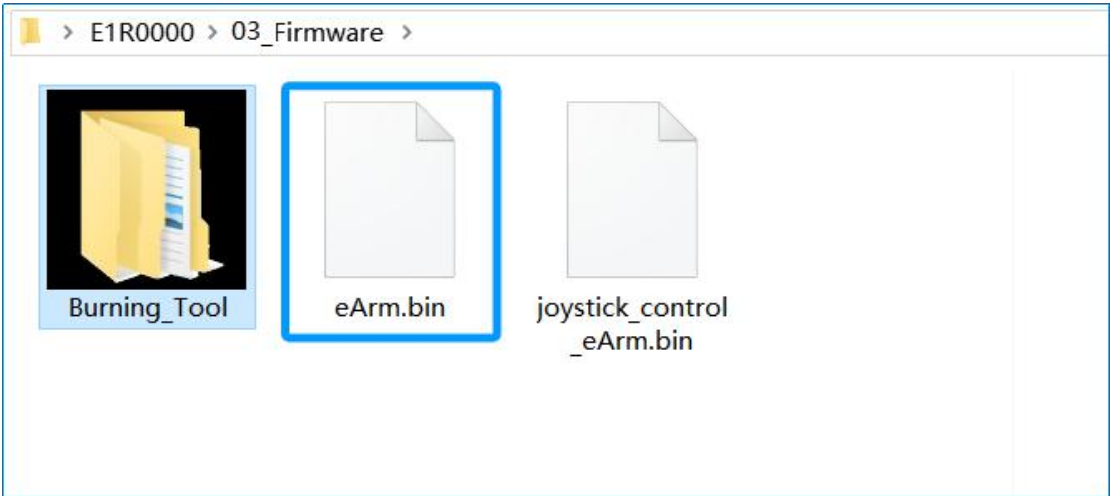
Flash Download Tools Expand all +

Please refer to Flash Download Tool User Guide to download this tool.

Title	Platform	Version	Release Date ▾	Download
+ Flash Download Tools	HTML	latest	2024.12.18	

2.2 eArm Firmware (Factory Firmware)

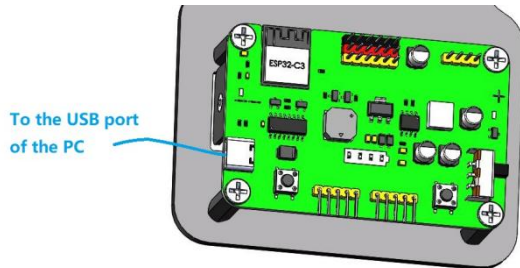
The functions of this firmware are exactly the same as the factory functions of the robotic arm. It also has the same functionality as the code in chapter 1.5.6. When the factory functions are lost, you can use this firmware to restore the factory settings. The firmware is saved in the following file:



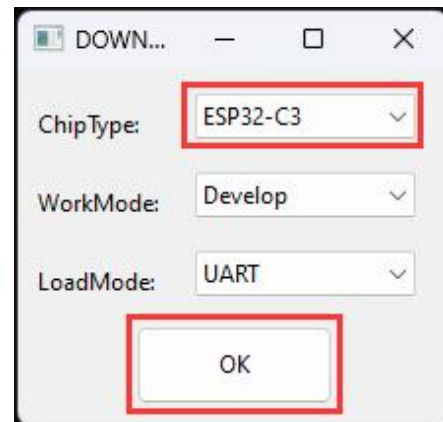
This firmware is generated from the code in Chapter 1.5.6!

2.2.1 Burning eArm Firmware

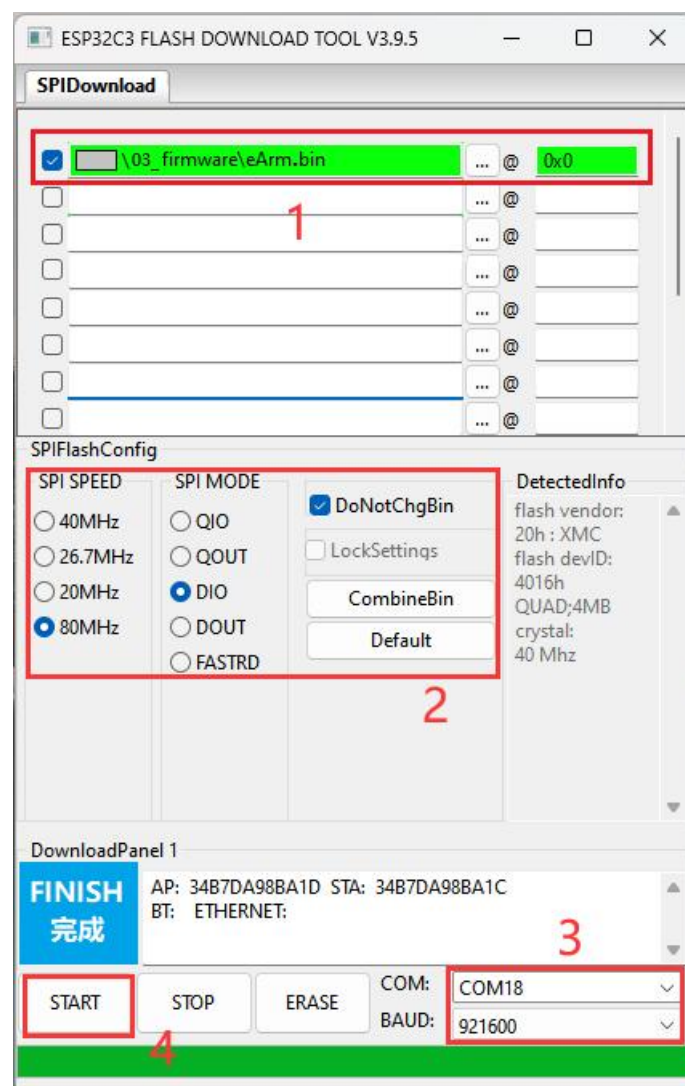
Use the USB cable to connect the eArm to PC:



Start the burning tool:



Select the firmware to be burned and correctly fill in the burning address.



The CH340 driver must be installed, the battery must be properly connected, and the power switch on the eArm must be turned on; otherwise, the COM port will not be detected.

1: Check one burning option  , then click  , and select the "eArm.bin" firmware file provided in our tutorial. Then, correctly enter the burning address (0x0) after "@":

File Name	Burning Address
eArm.bin	0x0

2: Select 80MHz for SPI SPEED, DIO for SPI MODE, and check  DotNotChgBin.

3: Select the serial port for the control board. Here, it is COM18 (Please select the port based on the actual port assigned to this device by your computer). Set the baud rate to 921600. If the burning fails, you can lower the baud rate, such as 115200.

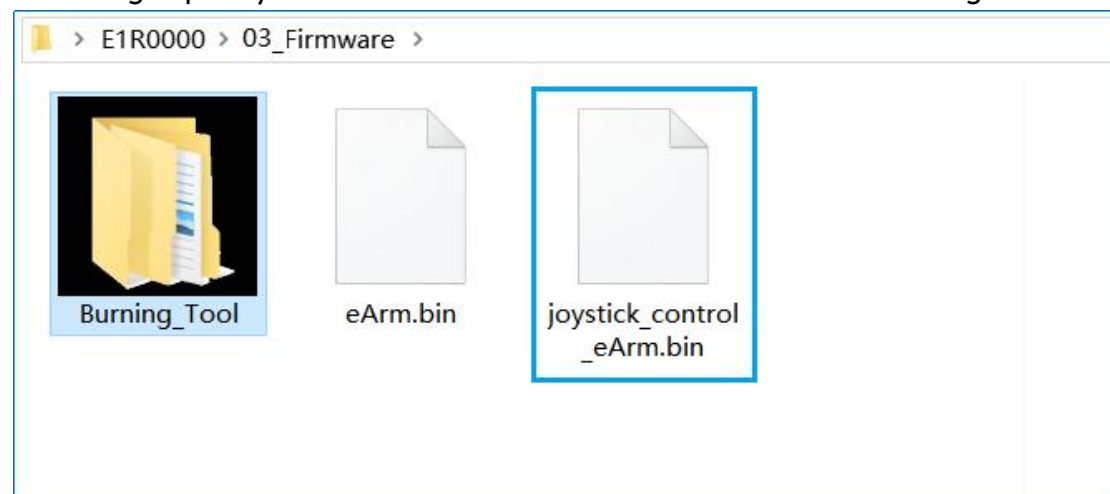
4: Click to start burning the firmware.

After burning is completed, you can follow section [Assembly and User Guide](#) to control the robotic arm!

Note: Once the firmware flashing is complete, you need to press the RST (Reset) button or disconnect the USB cable, and then power on.

2.3 Joystick Control Firmware (with Recording Function)

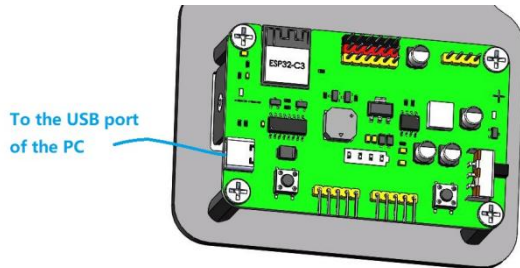
This firmware has the same functions as the code in chapter 1.5.4, with a recording capacity of 20 actions. The firmware is saved in the following file:



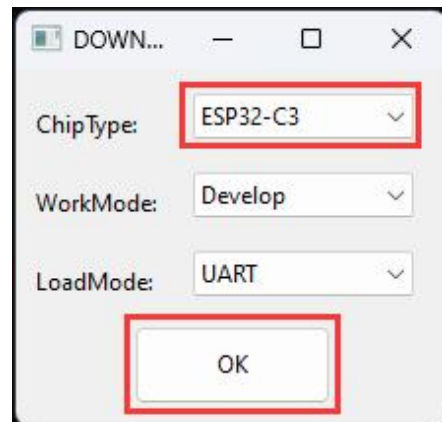
This firmware is generated from the code in Chapter 1.5.4

2.3.1 Burning Joystick Control Firmware

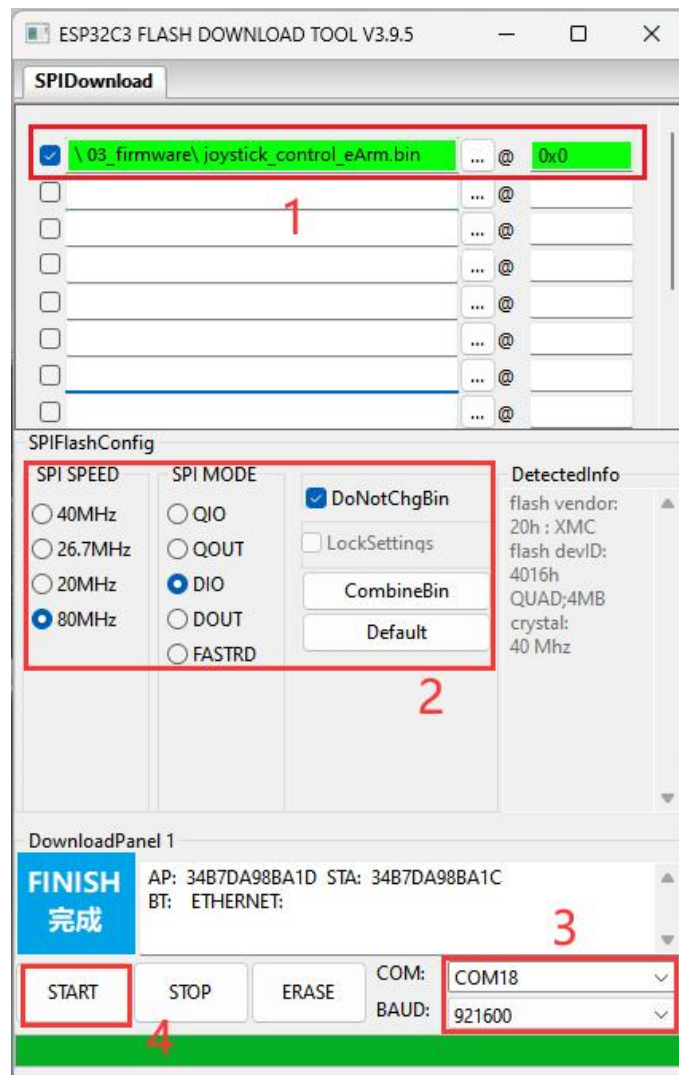
Use the USB cable to connect the eArm to PC:



Start the burning tool:



Select the firmware to be burned and correctly fill in the burning address



The CH340 driver must be installed, the battery must be properly connected, and the power switch on the eArm must be turned on; otherwise, the COM port will not be detected.

1: Check one burning option  , then click  , and select the "joystick_control_eArm.bin" firmware file provided in our tutorial. Then, correctly enter the burning address (0x0) after "@":

File Name	Burning Address
joystick_control_eArm.bin	0x0

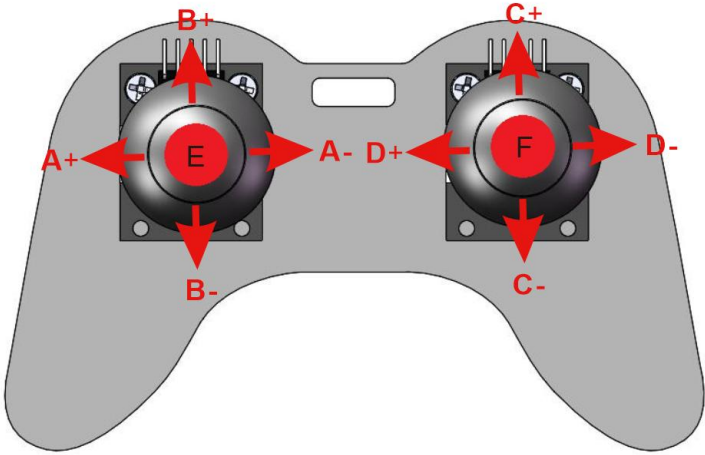
2: Select 80MHz for SPI SPEED, DIO for SPI MODE, and check  DotNotChgBin.

3: Select the serial port for the control board. Here, it is COM18 (Please select the port based on the actual port assigned to this device by your computer). Set the baud rate to 921600. If the burning fails, you can lower the baud rate, such as 115200.

4: Click to start burning the firmware.

Note: Once the firmware flashing is complete, you need to press the RST (Reset) button or disconnect the USB cable, and then power on.

Joystick Control - Robotic Arm Operation Guide:

	
A+: Controls Aservo, robotic arm rotates left.	A-: Controls Aservo, robotic arm rotates right.
B+ : Controls Bservo, robotic arm rear arm swings forward.	B-: Controls Bservo, robotic arm rear arm swings backward.
C+: Controls Cservo, robotic arm forearm swings forward.	C- : Controls Cservo, robotic arm forearm swings backward.
D+: Controls Dservo, robotic arm gripper opens (loosens).	D-: Controls Dservo, robotic arm gripper closes (tightens).
E: Record Action - Press E button once to record one action (up to 20 actions can be recorded in sequence). Each press triggers a short beep. When the maximum recording capacity is reached, the buzzer emits one long beep.	F : Playback — After recording your sequence, press the F button once to start looping the recorded actions continuously. The buzzer will beep once, and the arm will repeat the actions in a loop. To exit the loop, press and hold the F button (long press); the buzzer will emit one long beep to indicate playback has stopped.

3

QA

3.1 Unable to recognize USB serial ports

- 👉 Do you use a USB cable with data communication function?
- 👉 Does the USB cable connect well?
- 👉 Is the CH340 USB driver installed?

3.2 Robotic arm doesn't work

- 👉 Whether to turn on the power switch during installation to initialize the angle of the servo.
- 👉 Is the battery electricity sufficient?

3.3 Other problems

- 👉 Please check whether the assembly is correct?
- 👉 Please check whether the battery power is sufficient?
- 👉 Please check whether the battery used in accordance with the specifications?

4

Contact Us

If you encounter any technical issues or wish to share feedback with us, please feel free to contact us

✉ support@siyeenove.com

🌐 <https://siyeenove.com>